

CosmosDB

1. ¿Cómo funciona el particionamiento horizontal en ComosDB?

En Azure Cosmos DB, el particionamiento horizontal divide los datos de un contenedor (como una colección o tabla) en múltiples particiones de recursos, cada una basada en una clave de partición definida por el usuario. Estas particiones son unidades independientes y altamente disponibles que permiten distribuir los datos y las solicitudes de manera equilibrada.

El sistema gestiona automáticamente estas particiones, asegurando que el rendimiento, la consistencia y la disponibilidad no se vean afectados. Además, Cosmos DB permite escalar el rendimiento de forma elástica, ajustando el throughput (medido en Request Units o RUs) según la carga de trabajo y las regiones geográficas, garantizando que los cambios en la capacidad estén disponibles en pocos segundos en todas las ubicaciones distribuidas.

2. ¿Comente el modelo de consistencia de CosmosDB?

El modelo de consistencia de CosmosDB busca ofrecer un equilibrio entre rendimiento, disponibilidad y coherencia de los datos en un entorno globalmente distribuido. El sistema está diseñado para que los desarrolladores puedan escribir aplicaciones correctas y predecibles, evitando los errores comunes de los modelos de consistencia eventual.

CosmosDB proporciona niveles de consistencia bien definidos, permitiendo elegir entre opciones que van desde una consistencia fuerte hasta modelos más relajados. Estos modelos están formalmente especificados con TLA+, lo que garantiza propiedades comprobables y una programación más intuitiva. Además, el servicio mantiene baja latencia y alta disponibilidad (99.99%), incluso en múltiples regiones, y permite simular fallas regionales para validar la resistencia de las aplicación

3. ¿Comente como los objetivos de diseño de CosmosDB afectan la creación de aplicaciones?

Los objetivos de diseño de CosmosDB influyen directamente en la creación de aplicaciones al permitir configuraciones dinámicas que ajustan la proximidad entre el motor de base de datos y el almacenamiento, adaptándose a distintos niveles de servicio y rendimiento. Gracias a sus SLA definidos para rendimiento, latencia, consistencia y disponibilidad, los desarrolladores pueden construir aplicaciones confiables y con comportamiento predecible en entornos distribuidos.

Además, su diseño modular y gobernado por recursos facilita la replicación entre regiones y el escalamiento elástico del rendimiento sin afectar las garantías del sistema. Los modelos de consistencia formalmente especificados con TLA+ brindan a los programadores un marco claro para escribir aplicaciones distribuidas correctas. Finalmente, su motor de base de datos sin esquema y extensible, que soporta múltiples modelos y APIs, permite desarrollar una amplia variedad de aplicaciones de forma eficiente sobre una misma plataforma.